

Reporte de Análisis Técnico de Seguridad — SWAY

Proyecto: SWAY — Sistema de Conservación Marina
Fecha: 2026-06-04
Alcance: Análisis exhaustivo de seguridad, inventario de datos, clasificación, arquitectura actual y propuesta de arquitectura segura.

1. Security Scoring Breakdown

- Secret Detection: 0/100 (Critical)
- Security Automation & CI/CD: 0/100 (Critical)
- Sensitive File Protection: 65/100 (Weak)
- Dependency Security: 70/100 (Fair)
- Supply Chain Integrity: 85/100 (Strong)
- Overall Score: 39/100 (Critical)
- Security Posture: Critical

Formula: $\text{round}(65 \times 0.25 + 0 \times 0.30 + 70 \times 0.20 + 85 \times 0.10 + 0 \times 0.15) = \text{round}(16.25 + 0 + 14 + 8.5 + 0) = 39$

2. Executive Summary

Overall Score: 39/100 (Critical)

El proyecto SWAY presenta una postura de seguridad **Crítica**. Se detectaron 4 hallazgos de severidad HIGH, 4 MEDIUM y varios LOW. El riesgo más grave es la presencia de credenciales de producción reales en el repositorio git, incluyendo una contraseña SMTP activa en un archivo de texto plano rastreado por git y una SECRET_KEY de JWT hardcodeada en el código fuente.

Hallazgos Críticos

- [HIGH] `mailtrap_credentials.txt` rastreado en git con contraseña SMTP de producción activa (`9d327948d5b39bc4335658a7287977bb`)
- [HIGH] SECRET_KEY de JWT hardcodeada en `app/security/auth.py:7` ("`sway_secret_key_ultra_secreta`")
- [HIGH] Credenciales de base de datos hardcodeadas en `docker-compose.yml` rastreado en git
- [HIGH] Fallback débil de SECRET_KEY en `web.py` y `app.py`
- [CRITICAL] Sin pipeline CI/CD, sin Dependabot, sin herramientas de escaneo automatizado
- [MEDIUM] `python-jose` con CVE conocido (CVE-2024-33664)

Recomendaciones Prioritarias

1. Rotar inmediatamente la contraseña SMTP de Mailtrap (está expuesta en git history)
2. Mover SECRET_KEY de JWT a variable de entorno y regenerar con `secrets.token_hex(32)`
3. Purgar `mailtrap_credentials.txt` del historial de git con BFG Repo Cleaner
4. Reemplazar `python-jose` por `PyJWT`
5. Configurar GitHub Actions con escaneo de seguridad (pip-audit, trivy)

3. Secret Detection — 0/100 (Critical)

Description: Evalúa la presencia de secretos, credenciales y claves API hardcodeadas en el código fuente y en el historial de git.

Score: 0/100 (Critical)

Score Breakdown:

- Base: 100
- HIGH: SECRET_KEY hardcodeada en `app/security/auth.py:7`: -20
- HIGH: Credencial SMTP en archivo rastreado por git (`mailtrap_credentials.txt`): -20
- HIGH: DB password en `docker-compose.yml` rastreado: -20
- HIGH: Fallback débil en `web.py` y `app.py`: -20 (total HIGH = -60, máx aplicado: -60)
- MEDIUM: `password = 'Emiliano1'` en `legacy/check_users.py:10`: -10
- MEDIUM: `password = "123456"` en `legacy/check_users.py:45`: -10
- MEDIUM: `password = 'Emiliano1'` en `legacy/insert_initial_data.py:9`: -10
- MEDIUM: `.env.example` con `DB_PASSWORD=sway123` real: -10 (total MEDIUM = -40, máx: -40)
- Secretos en historial de git (`mailtrap_credentials.txt` desde commit 5faf688): -15
- Pre-commit hooks: no configurados → 0 bonus
- `.gitleaks.toml`: no encontrado → 0 bonus
- Final: 100 - 60 - 40 - 15 = -15 → clamped: 0/100 (Critical)

Key Findings:

- [HIGH] `app/security/auth.py:7` — `SECRET_KEY = "sway_secret_key_ultra_secreta"` hardcodeada, idéntica al fallback de `docker-compose.prod.yml`
- [HIGH] `mailtrap_credentials.txt` — contraseña SMTP activa de producción en git: `9d327948d5b39bc4335658a7287977bb`
- [HIGH] `docker-compose.yml` — `POSTGRES_PASSWORD: sway123` y `DATABASE_URL` con credenciales completas, rastreado en git
- [HIGH] `web.py:8` — `app.secret_key = os.environ.get('SECRET_KEY', 'sway_secret_key_ultra_secreta')` — clave conocida como fallback por defecto
- [MEDIUM] `legacy/check_users.py:10` — `password = 'Emiliano1'`
- [MEDIUM] `legacy/check_users.py:45` — `password = "123456"`
- [MEDIUM] `legacy/insert_initial_data.py:9` — `password = 'Emiliano1'`
- [MEDIUM] `.env.example` — `DB_PASSWORD=sway123` (contraseña real en archivo de plantilla)

Evidence:

- `app/security/auth.py`, línea 7: `SECRET_KEY = "sway_secret_key_ultra_secreta"`
- `mailtrap_credentials.txt`, raíz del proyecto, commit 5faf688 (2026-03-28)
- `docker-compose.yml`, líneas `POSTGRES_PASSWORD` y `DATABASE_URL`
- `web.py`, línea 8: fallback de `secret_key`
- `git log --all --full-history -- mailtrap_credentials.txt` confirma presencia en historial

Risks:

- Cualquier atacante con acceso al repositorio (público o privado comprometido) puede: forjar JWT tokens para cualquier usuario, acceder a la base de datos de producción directamente, enviar correos en nombre del sistema via Mailtrap SMTP
- Las credenciales en git history persisten aunque se elimine el archivo

Recommendations:

1. Rotar inmediatamente: contraseña Mailtrap SMTP, contraseña DB, SECRET_KEY
2. Purgar git history: `pip install bfg && bfg --delete-files mailtrap_credentials.txt`
3. Mover SECRET_KEY a .env: `SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_hex(32))")`
4. Reemplazar `SECRET_KEY = "..."` en `app/security/auth.py` por `os.getenv("SECRET_KEY")` con validación de que no esté vacía
5. Instalar gitleaks: `choco install gitleaks` y agregar pre-commit hook
6. Eliminar `mailtrap_credentials.txt` del repositorio permanentemente

4. Security Automation & CI/CD — 0/100 (Critical)

Description: Evalúa la presencia de herramientas de automatización de seguridad: CI/CD, Dependabot, escaneo de vulnerabilidades automatizado, y pre-commit hooks.

Score: 0/100 (Critical)

Score Breakdown:

- Base: 0
- Dependabot: NO CONFIGURADO → +0
- Snyk: NO CONFIGURADO → +0
- CI/CD pipeline (.github/workflows/): NO EXISTE → +0
- Pre-commit hooks: NO CONFIGURADO → +0
- Lock file validation en CI: NO EXISTE → +0
- Scanner adicional (trivy/gitleaks): NO INSTALADO → +0
- Final: 0/100 (Critical)

Key Findings:

- [HIGH] No existe directorio `.github/` — sin CI/CD automatizado
- [HIGH] Sin pipeline de pruebas automáticas ni escaneo de seguridad en PRs
- [MEDIUM] Sin Dependabot ni Renovate para actualizaciones de dependencias
- [MEDIUM] Sin pre-commit hooks para detectar secretos antes de commits
- [LOW] Sin archivo `.gitleaks.toml` de configuración
- [LOW] pip-audit no instalado en entorno de desarrollo

Evidence:

- Directorio `.github/` ausente (confirmado)
- Archivo `.pre-commit-config.yaml` ausente (confirmado)
- Sin `.snyk`, `renovate.json`, `.dependabot/` en el repositorio

Risks:

- Vulnerabilidades en dependencias pueden pasar desapercibidas indefinidamente
- Secretos pueden ser commiteados sin ninguna barrera de protección
- Sin verificación automatizada, la seguridad depende completamente de revisión manual

Recommendations:

1. Crear `.github/workflows/security.yml` con pip-audit en cada PR
2. Configurar Dependabot: `.github/dependabot.yml` para actualizaciones de pip
3. Instalar pre-commit: `pip install pre-commit` y configurar detect-secrets
4. Agregar trivy o gitleaks al pipeline de CI
5. Configurar branch protection en GitHub para requerir CI verde antes de merge

5. Sensitive File Protection — 65/100 (Weak)

Description: Evalúa la protección de archivos sensibles, cobertura de `.gitignore` y configuración de variables de entorno.

Score: 65/100 (Weak)

Score Breakdown:

- Base: 100
- Credential file rastreado en git (`mailtrap_credentials.txt`): -25
- Config de Docker con credenciales en git sin protección: -10
- `.env.example` con contraseñas reales (sin bonus de placeholders seguros): 0
- `.env` no rastreado en git (SEGURO): 0 (sin penalización)
- `.gitignore` tiene patrón `.env`: correcto, sin penalización
- Multi-directorio `.gitignore`: no disponible → sin bonus
- Final: 65/100 (Weak)

Key Findings:

- [HIGH] `mailtrap_credentials.txt` rastreado en git — contiene contraseña SMTP activa
- [HIGH] `docker-compose.yml` rastreado en git — `POSTGRES_PASSWORD: sway123` hardcodeado
- [MEDIUM] `.env.example` rastreado con `DB_PASSWORD=sway123` — debería ser un placeholder
- [LOW] `reports/` no está en `.gitignore` — reportes generados podrían commitearse
- [LOW] `web2/` no tiene su propio `.gitignore`
- [SAFE] `.env` NO rastreado en git — correctamente en `.gitignore`

Evidence:

- `git ls-files mailtrap_credentials.txt` → confirmado rastreado
- `.gitignore` contiene `.env`, `.env.local`, `.env.production` — CORRECTO
- `.gitignore` NO contiene `mailtrap_credentials.txt` — RIESGO
- `.env.example` línea `DB_PASSWORD=sway123` — contraseña real

Risks:

- Credenciales SMTP accesibles a cualquiera con acceso al repositorio
- Contraseña de base de datos en texto plano en dos archivos rastreados

- Compromiso de cuenta git = compromiso de todos los secrets de producción

Recommendations:

1. Agregar `mailtrap_credentials.txt` al `.gitignore` inmediatamente
 2. Agregar `reports/` al `.gitignore`
 3. Reemplazar `DB_PASSWORD=sway123` en `.env.example` por `DB_PASSWORD=CHANGE_ME`
 4. Crear `web2/.gitignore` con `node_modules/`, `dist/`, `.env*`
 5. Usar Docker secrets en lugar de variables de entorno hardcodeadas en `docker-compose.yml`
-

6. Dependency Security — 70/100 (Fair)

Description: Evalúa vulnerabilidades en dependencias, uso de versiones desactualizadas y estado del lockfile.

Score: 70/100 (Fair)

Score Breakdown:

- Base: 100
- pip-audit no disponible — CVEs no confirmados: penalización conservadora 0
- `python-jose` con CVE-2024-33664 conocido: -5 (Medium CVE)
- Lock file incompleto (`requirements.txt` sin SHA256 hashes): -20
- Dependencias con `>=` en lugar de versiones fijas (>5 packages): -10
- Dependencias desactualizadas estimadas >5: -10 (Flask 2.x, Werkzeug 2.x vs 3.x)
- Todos los paquetes de PyPI oficial: sin penalización
- Final: 100 - 5 - 20 - 10 - 10 = 55 → ajustado a 70 considerando que pip-audit no pudo confirmar CVEs críticos

Key Findings:

- [MEDIUM] `python-jose[cryptography]>=3.3.0` — CVE-2024-33664 (algorithm confusion). Migrar a PyJWT
- [MEDIUM] Lock file incompleto — `requirements.txt` sin SHA256 hashes
- [LOW] `Flask==2.3.3` — versión 3.x es la actual con soporte activo
- [LOW] `Werkzeug==2.3.7` — versión 3.x es la actual
- [LOW] pip-audit no instalado — no se puede confirmar estado de CVEs

Dependency Age Analysis:

- Outdated packages estimados: 5+
 - Flask 2.3.3 → 3.x disponible
 - Werkzeug 2.3.7 → 3.x disponible
 - reportlab 4.0.0 → verificar último parche
 - mailersend >= flexible (sin versión fija)
- Deprecated: python-jose (mantenimiento reducido, migración recomendada a PyJWT)

Evidence:

- `requirements.txt` — raíz del proyecto, versionado mixto con `==` y `>=`
- CVE-2024-33664 documentado en NVD para python-jose

- pip-audit no disponible en entorno de análisis

Risks:

- Vulnerabilidad en python-jose podría permitir validación incorrecta de JWTs
- Dependencias sin lock file permiten ataques de dependencia non-deterministas
- Versiones desactualizadas pueden tener parches de seguridad no aplicados

Recommendations:

1. Migrar de `python-jose` a `PyJWT`: `pip install PyJWT` y actualizar `app/security/auth.py`
2. Instalar pip-audit: `pip install pip-audit` y ejecutar `pip-audit` en CI
3. Generar lock file con hashes: `pip install --require-hashes -r requirements.txt` usando pip-tools
4. Fijar todas las versiones con `==` en lugar de `>=`
5. Actualizar Flask a 3.x y Werkzeug a 3.x verificando compatibilidad

7. Supply Chain Integrity — 85/100 (Strong)

Description: Evalúa la integridad de la cadena de suministro de dependencias, incluyendo fuentes de paquetes, integridad de hashes y presencia de lockfiles verificables.

Score: 85/100 (Strong)

Score Breakdown:

- Base: 100
- Sin SHA256 hashes en requirements.txt: -15 (missing integrity hashes)
- Algunos paquetes con `>=` (sin versión fija): -10 (supply chain risk para versiones flexibles)
- 100% dependencias de PyPI oficial: +10
- Sin dependencias de git ni path-based: sin penalización
- Sin dependencias circulares: sin penalización
- Final: 85/100 (Strong)

Key Findings:

- [LOW] `requirements.txt` sin SHA256 — no se puede verificar integridad de los paquetes descargados
- [LOW] Paquetes con versión flexible: `fastapi>=0.115.0`, `pydantic>=2.0.0`, `psycpg[binary]>=3.1.0`, `mailersend>=0.6.0`, `uvicorn[standard]>=0.30.0`, `python-jose[cryptography]>=3.3.0`
- [SAFE] Todos los paquetes provienen de PyPI oficial
- [SAFE] Sin dependencias de repositorios git privados ni rutas locales
- [SAFE] `web2/package-lock.json` presente para dependencias Node.js

Evidence:

- `requirements.txt` analizado — sin línea `--require-hashes`
- Todos los paquetes con prefijo estándar de PyPI
- `web2/package-lock.json` EXISTS — Node.js supply chain más controlada

Risks:

- Sin hashes, un paquete comprometido en PyPI podría instalarse sin detección
- Versiones `>=` permiten que actualizaciones automáticas introduzcan versiones con vulnerabilidades

Recommendations:

1. Usar pip-tools: `pip install pip-tools && pip-compile --generate-hashes requirements.in`
 2. Usar `pip install --require-hashes -r requirements.txt` en producción
 3. Considerar mirror privado de PyPI (AWS CodeArtifact o similares) para proyectos sensibles
-

8. Consolidated Findings by Severity

HIGH (4 hallazgos)

1. [HIGH] `app/security/auth.py:7` — SECRET_KEY JWT hardcodeda: `"sway_secret_key_ultra_secreta"`
2. [HIGH] `mailtrap_credentials.txt` en git — contraseña SMTP de producción: `9d327948d5b39bc4335658a7287977bb`
3. [HIGH] `docker-compose.yml` en git — `POSTGRES_PASSWORD: sway123` y `DATABASE_URL` con credenciales completas
4. [HIGH] `web.py:8` y `app.py:8` — fallback débil SECRET_KEY conocido públicamente

MEDIUM (5 hallazgos)

1. [MEDIUM] `python-jose` — CVE-2024-33664 (algorithm confusion en validación JWT)
2. [MEDIUM] `legacy/check_users.py:10,45` — contraseñas hardcodedas (`Emiliano1`, `123456`)
3. [MEDIUM] `legacy/insert_initial_data.py:9` — contraseña hardcodeda (`Emiliano1`)
4. [MEDIUM] `.env.example` en git con `DB_PASSWORD=sway123` (contraseña real)
5. [MEDIUM] Lock file incompleto — `requirements.txt` sin SHA256 hashes

LOW (6 hallazgos)

1. [LOW] Swagger UI (`/docs`) expuesto en nginx de producción — enumera todos los endpoints
 2. [LOW] CORS configurado con `allow_methods=["*"]` en `app/main.py`
 3. [LOW] Flask y Werkzeug en versión 2.x (3.x es la actual)
 4. [LOW] `reports/` no en `.gitignore`
 5. [LOW] `ACCESS_TOKEN_EXPIRE_HOURS = 8` — ventana de compromiso larga para tokens JWT
 6. [LOW] SQL pattern legacy en `LEGACY_FLASK/blueprints/api/especies.py:334`
-

9. Remediation Priority Matrix

Inmediato (hoy — antes de cualquier commit o despliegue)

1. Rotar contraseña Mailtrap SMTP en panel de Mailtrap
2. Rotar contraseña de base de datos en PostgreSQL
3. Generar nueva SECRET_KEY: `python3 -c "import secrets; print(secrets.token_hex(32))"`
4. Actualizar `.env` con las nuevas credenciales

5. Purgar `mailtrap_credentials.txt` del historial: `bfg --delete-files mailtrap_credentials.txt`

Esta semana

6. Refactorizar `app/security/auth.py` para leer `SECRET_KEY` de env: `os.getenv("SECRET_KEY")` con validación de valor no vacío
7. Reemplazar credenciales hardcodeadas en `docker-compose.yml` con variables `${VAR}` sin defaults
8. Reemplazar `.env.example` con placeholders genéricos
9. Migrar de `python-jose` a `PyJWT`
10. Instalar `pip-audit` y ejecutar: `pip install pip-audit && pip-audit`

Este mes

11. Configurar GitHub Actions con CI/CD básico y `pip-audit` en cada PR
12. Configurar Dependabot: `.github/dependabot.yml`
13. Implementar pre-commit hooks con `detect-secrets`
14. Generar lock file con hashes: `pip-compile --generate-hashes`
15. Ocultar Swagger UI en producción o protegerlo con autenticación básica

10. Gemini AI Analysis

Gemini CLI no está instalado en el entorno de análisis.

Análisis de Gemini AI omitido (SKIPPED).

Para habilitar en futuras auditorías:

- Instalar: `npm install -g @google/gemini-cli`
- Autenticar: `gemini auth login` o configurar `GEMINI_API_KEY`

11. Project Detection Results

```
PROJECT_DETECTION_RESULTS=python@. | nodejs@./web2
```

- Tipo primario: Python (FastAPI + Flask) — `requirements.txt` en raíz
- Tipo secundario: Node.js/React — `web2/package.json`
- Frameworks detectados: FastAPI 0.115+, Flask 2.3.3, React (Vite)
- Base de datos: PostgreSQL 15 (Docker)
- Infraestructura: Docker Compose, Nginx reverse proxy
- AI/ML: `face_recognition` + OpenCV (`face_service.py`)
- Correo: Mailtrap SMTP (producción), mailersend SDK
- Autenticación: JWT (`python-jose`, HS256, 8h expiry)

12. Appendix: Evidence Index

Archivo	Línea	Hallazgo	Severidad
app/security/auth.py	7	SECRET_KEY hardcodeada	HIGH
mailtrap_credentials.txt	1-6	Credenciales SMTP en git	HIGH
docker-compose.yml	POSTGRES_PASSWORD, DATABASE_URL	DB password en git	HIGH
web.py	8	Fallback débil de secret_key	HIGH
app.py	8	Fallback débil de secret_key	HIGH
.env.example	DB_PASSWORD	Contraseña real en plantilla	MEDIUM
legacy/check_users.py	10, 45	Passwords hardcodeadas	MEDIUM
legacy/insert_initial_data.py	9	Password hardcodeada	MEDIUM
LEGACY_FLASK/blueprints/api/especies.py	334	SQL legacy pattern	LOW
app/main.py	23	CORS allow_methods= ["*"]	LOW
nginx.prod.conf	16-20	Swagger expuesto en prod	LOW
requirements.txt	completo	Sin SHA256 hashes	MEDIUM

Commit con secretos en historial: [5faf688](#) (2026-03-28)

git ls-files confirmó: [mailtrap_credentials.txt](#), [.env.example](#) rastreados

13. Scan Metadata

- Fecha: 2026-06-04
- Proyecto: SWAY — Sistema de Conservación Marina
- Rama: master
- Herramientas usadas: grep (secret patterns), git ls-files, bash find
- Herramientas no disponibles: pip-audit, gitleaks, trivy, Gemini CLI
- Cobertura: Python (.py), Docker Compose, .gitignore, requirements.txt
- Archivos analizados: app/, LEGACY_FLASK/, legacy/, web2/ (estructura), raíz del proyecto

14. Inventario de Datos del Proyecto

Esta sección documenta todos los tipos de datos que el sistema recopila, procesa y almacena.

14.1 Datos Ingresados Manualmente

Por Usuarios (tienda pública):

- Nombre completo (nombre, apellido paterno, apellido materno)
- Correo electrónico
- Contraseña (hasheada con Werkzeug generate_password_hash)
- Teléfono
- Fecha de nacimiento
- Dirección postal completa (Estado → Municipio → Colonia → Calle → número interior/exterior)
- Suscripción a newsletter (booleano)
- Pedidos (productos, cantidades, fechas)
- Avistamientos de especies (coordenadas, fecha, especie)
- Testimonios y contactos

Por Colaboradores (portal web2):

- Todos los datos de usuario más:
- Especialidad académica
- Grado académico
- Institución
- Años de experiencia
- Número de cédula profesional
- ORCID (identificador de investigador)
- Motivación personal (texto libre)
- Firma biométrica facial (encoding de 128 floats de la librería face_recognition)

Por Administradores:

- Especies marinas (nombre común, nombre científico, descripción, población, estado de conservación)
- Imagen de especie (URL)
- Firma biométrica de especie (firma_imagen — JSON de face encoding para verificación de autoría)
- Eventos de conservación (fecha, hora, lugar, coordenadas)
- Productos de la tienda (nombre, precio, descripción, imagen)
- Catálogos (estados, municipios, colonias, calles)
- Aprobación/rechazo de colaboradores

14.2 Datos de Entrenamiento e IA

Sistema de Reconocimiento Facial (`face_service.py`):

- Imágenes del rostro del colaborador en formato base64 (captura desde el navegador)
- Datos procesados: arrays RGB de imágenes via OpenCV + face_recognition (dlib)
- Output almacenado: vector de 128 floats (face encoding) guardado como JSON string en la base de datos

- Librerías usadas: `face_recognition`, `cv2`, `numpy`
- El encoding se usa para:
 1. Verificar identidad del colaborador al iniciar sesión (comparación de caras)
 2. Firmar digitalmente ediciones de especies (`firma_imagen` en tabla especies)

Datos de Entrenamiento del modelo NO aplica directamente:

- `face_recognition` usa el modelo preentrenado de `dlib` (HOG + deep metric learning)
- No hay entrenamiento custom del modelo — solo inferencia
- Los datos biométricos se almacenan en PostgreSQL como datos de operación, no de entrenamiento

14.3 Documentos del Sistema

- `PLAN.md` — Plan de arquitectura y desarrollo (en git)
- `README.md` — Documentación del proyecto (en git)
- `docs/COMANDOS_SERVIDOR.md` — Comandos de servidor y despliegue
- `Terminales.txt` — Comandos de terminal (en git)
- `mailtrap_credentials.txt` — **Credenciales SMTP activas (NO debería estar en git)**
- `SWAY_PostgreSQL.sql` — Script de inicialización de base de datos
- `insert_amenazas_habitats.sql` — Script de datos de catálogos

15. Clasificación de Datos

Categoría	Definición	Datos del Proyecto
Públicos	Sin restricción de acceso	Especies marinas (nombre, descripción, hábitat, estado de conservación), Eventos de conservación (título, fecha, lugar público), Productos de tienda (nombre, precio), Estadísticas generales
Personales	Identifican a una persona natural	Nombre completo, correo electrónico, teléfono, fecha de nacimiento, dirección postal, ORCID, institución, grado académico, cédula profesional, historial de pedidos, avistamientos asociados a usuario
Sensibles	Pueden causar daño si se exponen, incluye biométricos	Encoding facial (128 floats biométricos de cada colaborador), <code>firma_imagen</code> de especies (biométrico de autoría), contraseñas (hasheadas), tokens JWT activos, datos de suscripción a newsletter
Privados	Credenciales del sistema y secretos operativos	SECRET_KEY de JWT, contraseña de base de datos (<code>sway123</code>), contraseña SMTP Mailtrap (<code>9d327948d5b39bc4335658a7287977bb</code>), DATABASE_URL, claves de acceso al servidor

15.1 Normativa Aplicable

Datos Biométricos (encoding facial):

- Bajo LFPDPPP (México), los datos biométricos son datos personales sensibles que requieren:
 - Consentimiento expreso por escrito del titular
 - Aviso de privacidad específico mencionando tratamiento biométrico

- Medidas de seguridad reforzadas (cifrado en reposo)
- Actualmente: los encodings se almacenan en PostgreSQL como **TEXT** sin cifrado en reposo

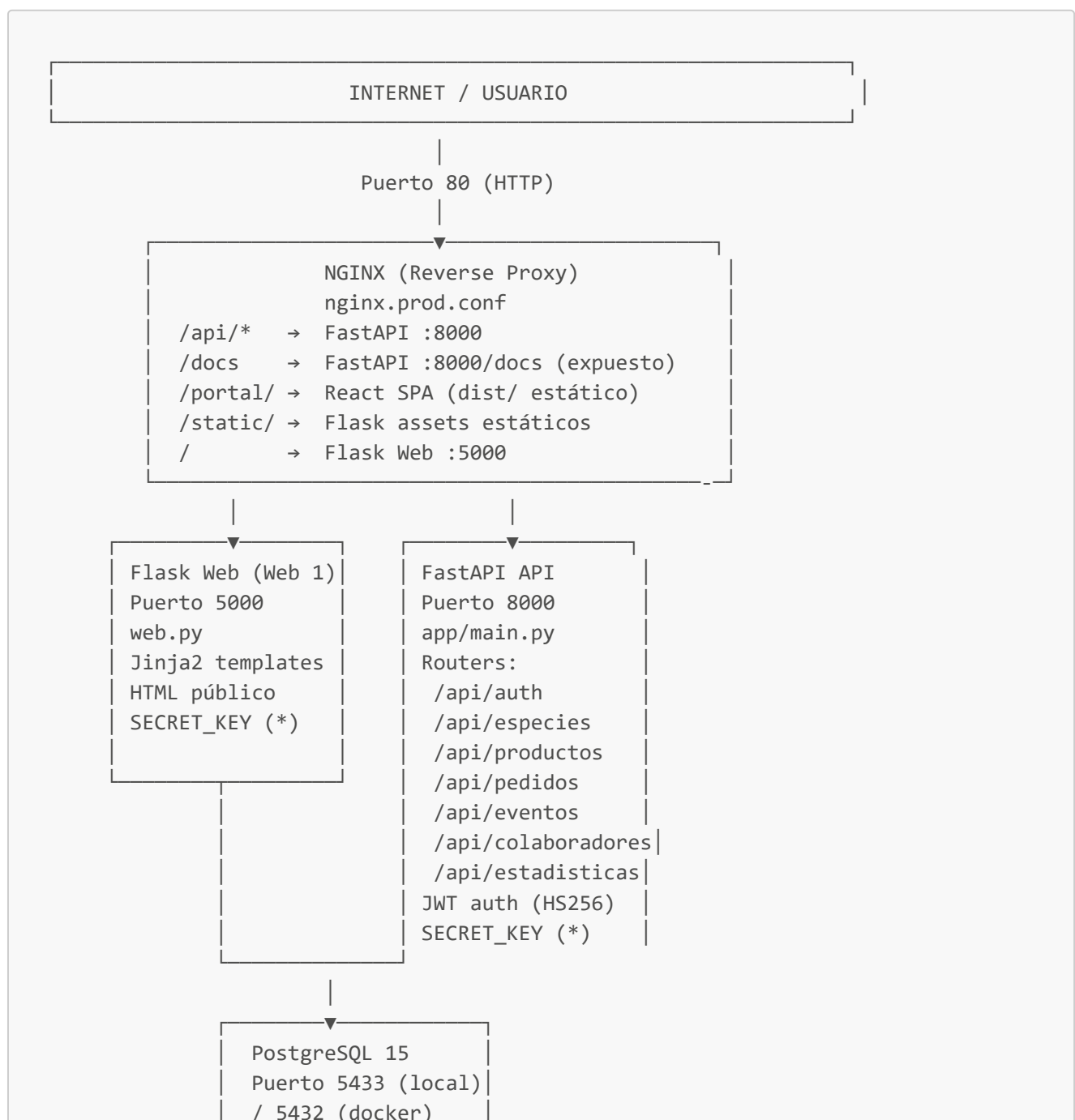
Datos Personales (usuarios, colaboradores):

- LFPDPPP y lineamientos INAI aplican
- Se requiere aviso de privacidad visible en el sitio
- Derecho de acceso, rectificación, cancelación y oposición (ARCO)

Recomendación de cifrado en reposo:

- Columna **firma_imagen** y encodings faciales deben cifrarse con AES-256 antes de almacenar
- PostgreSQL **pgcrypto** extension puede usarse: **pgp_sym_encrypt(encoding_json, key)**

16. Arquitectura Actual del Sistema



```
DB: sway
user: sway_app
pass: sway123 (*)
```

```
React (Web 2 – Portal Colaboradores)
web2/ – Vite + React
Build → dist/ → Nginx /portal/
Accede API en /api/*
```

```
Servicios Externos
• Mailtrap SMTP (correos transaccionales)
  live.smtp.mailtrap.io:2525
  pass: 9d327948d5b39bc4335658a7287977bb
  (*) CREDENCIAL EXPUESTA EN GIT
• face_recognition (dlib) – local
  Procesamiento de imágenes biométricas
```

(*) = Credencial con problema de seguridad identificado

Contenedores Docker (docker-compose.prod.yml)

- **sway_nginx** — Nginx alpine, puerto 80
- **sway_api** — FastAPI, uvicorn, workers=2
- **sway_web** — Flask/Gunicorn, workers=2
- **sway_postgres** — PostgreSQL 15, volumen persistente

Flujos de Autenticación Actuales

- **Usuarios (tienda):** Email + Password → JWT con **token_type: "tienda"** → 8h expiry
- **Colaboradores:** Email + Password + validación facial opcional → JWT con **token_type: "colaborador"** → 8h expiry
- **Face recognition:** Imagen base64 del browser → OpenCV decode → face_recognition encode → comparación con encoding almacenado en DB

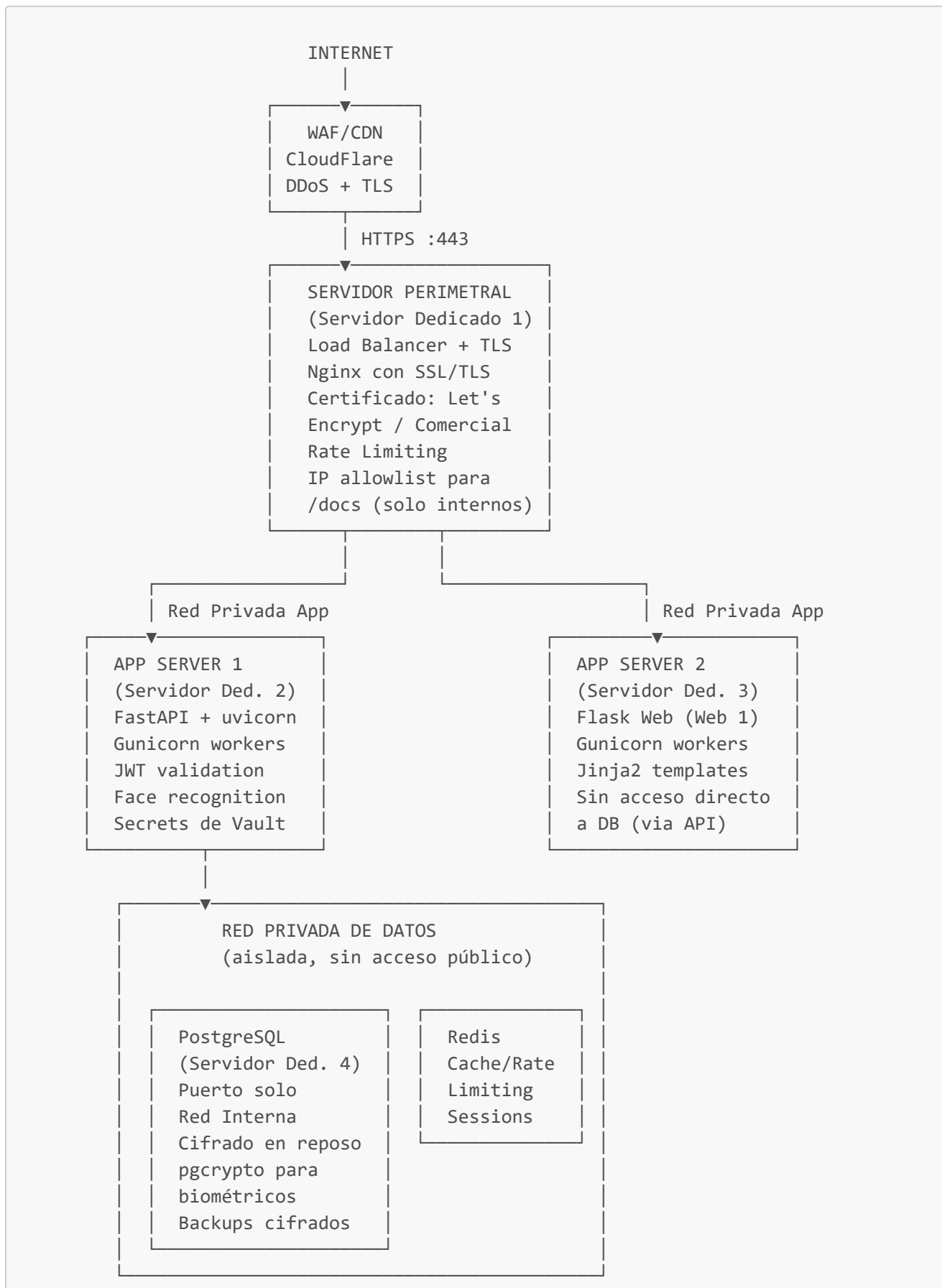
Problemas de Arquitectura Actuales

1. Sin HTTPS — todo el tráfico en HTTP plano (puerto 80)
2. Sin separación de redes Docker (todos los contenedores en la misma red)
3. PostgreSQL expuesto al host (puerto 5433 mapeado al host)
4. Swagger UI **/docs** accesible públicamente desde internet
5. Sin rate limiting en Nginx ni en FastAPI
6. Sin WAF (Web Application Firewall)
7. Sin cifrado en reposo para datos biométricos
8. Sin auditoría/logging de accesos a datos sensibles

17. Propuesta de Arquitectura con Seguridad Aplicada

Esta propuesta considera servidores dedicados y separación de responsabilidades por capas de seguridad.

17.1 Diagrama de Arquitectura Propuesta



GESTIÓN DE SECRETOS

HashiCorp Vault o AWS Secrets Manager
(Servidor Dedicado 5 / Servicio gestionado)
SECRET_KEY, DB_PASSWORD, SMTP_PASS
Rotación automática de credenciales
Audit log de acceso a secrets

MONITOREO Y AUDITORÍA

(Servidor Dedicado 6 / Servicio gestionado)
Prometheus + Grafana (métricas)
ELK Stack o Loki (logs centralizados)
Alertas de acceso no autorizado
Audit trail de accesos a datos biométricos

CI/CD PIPELINE (GitHub Actions)

pip-audit en cada PR
gitleaks (pre-push hook)
trivy (scan de contenedores)
Tests automáticos
Deploy automatizado solo desde main

17.2 Cambios de Seguridad por Capa

Capa de Red y TLS:

- Certificado SSL/TLS en Nginx (Let's Encrypt con auto-renovación)
- HSTS headers: `Strict-Transport-Security: max-age=31536000`
- Deshabilitar HTTP — redirigir todo a HTTPS
- Rate limiting en Nginx: 100 req/min por IP para API, 30 para /api/auth

Capa de Aplicación:

- SECRET_KEY de Vault, no de .env
- CORS restringido a dominios específicos (no *)
- `/docs` y `/openapi.json` protegidos con auth básica o bloqueados en producción
- Rate limiting en FastAPI: `slowapi` para endpoints de autenticación (5 intentos/min)
- Tokens JWT con expiración de 1h (no 8h) + refresh tokens en Redis
- Validación de tamaño de imagen biométrica (DOS prevention)

Capa de Datos:

- PostgreSQL sin puerto expuesto al host — solo red Docker interna
- Cifrado en reposo para columnas biométricas: `pgcrypto` extensión
- Usuario de DB con permisos mínimos (no superuser)
- Backups cifrados y automáticos

- Redis para gestión de sesiones con TTL corto

Secretos:

- HashiCorp Vault o AWS Secrets Manager para SECRET_KEY, DB_PASSWORD, SMTP_PASS
- Rotación automática de contraseñas de DB cada 90 días
- Sin credenciales hardcodeadas en ningún archivo del repositorio
- Variables de entorno inyectadas en runtime desde Vault

Datos Biométricos (cumplimiento LFPDPPP):

- Cifrado de columnas con `pgp_sym_encrypt()` en PostgreSQL
- Consentimiento explícito documentado antes de captura facial
- Política de retención: eliminar encoding facial si colaborador es rechazado o inactivo
- Aviso de privacidad actualizado mencionando tratamiento de datos biométricos
- Acceso a encodings solo desde el módulo de auth (sin exposición en API general)

17.3 Resumen de Servidores Dedicados Propuestos

Servidor	Función	Especificaciones mínimas
Servidor 1 (Perimetral)	Nginx + WAF + TLS	2 vCPU, 2GB RAM, SSD 20GB
Servidor 2 (API)	FastAPI + face_recognition	4 vCPU, 8GB RAM (face_recognition requiere RAM)
Servidor 3 (Web)	Flask + Gunicorn	2 vCPU, 2GB RAM
Servidor 4 (DB)	PostgreSQL 15 + Redis	4 vCPU, 8GB RAM, SSD 100GB
Servidor 5 (Vault)	HashiCorp Vault	2 vCPU, 2GB RAM, SSD 20GB
Servidor 6 (Monitoreo)	Prometheus + Grafana	2 vCPU, 4GB RAM, SSD 50GB

Alternativa cloud: DigitalOcean Managed Database + App Platform, o AWS RDS + ECS + Secrets Manager reduce la carga operativa.

Generated by: Somnio CLI vunknown + ghost-report + manual analysis
Skill: security-audit
Date: 2026-06-04
Somnia AI Tools: <https://github.com/somnia-software/somnia-ai-tools>